TD3 — Conditions et boucles en Bash

Systèmes d'exploitation — L1 Mathématiques / Informatique

Axel Guérin

Objectifs du TD

- Utiliser `if`, `elif` et `else`.
- Comparer des nombres et des chaînes.
- Vérifier l'existence de fichiers et les arguments.
- Répéter des opérations avec `for` et `while`.
- Automatiser des traitements Python.

Objectif

Construire un script capable de prendre des décisions et de traiter plusieurs fichiers.

Rappel du TD2

Un script Bash permet d'enchaîner des commandes.

```
#!/bin/bash
nom="$1"
echo "Bonjour $nom"
python3 programme.py "$nom"
```

Que faire si aucun argument n'est donné ?

Pourquoi des conditions ?

Un script doit parfois adapter son comportement.

- Un fichier est-il présent ?
- L'utilisateur a-t-il fourni les arguments attendus ?
- Un traitement a-t-il réussi ?

Principe

Une condition permet d'exécuter un bloc uniquement lorsqu'un test réussit.

Première condition : if

```
age=18
if [ "$age" -ge 18 ]; then
echo "Majeur"
fi
```

- `if` : début du test.
- `then` : début des commandes.
- `fi` : fin du bloc.

Comparaisons numériques

Opérateur	Signification
`-eq`	égal
`-ne`	différent
`-lt`	inférieur
`-le`	inférieur ou égal
`-gt`	supérieur
`-ge`	supérieur ou égal

Alternative : else

```
#!/bin/bash
note="$1"
if [ "$note" -ge 10 ]; then
echo "Admis"
else
echo "Non admis"
fi
```

Essayez : `bash resultat.sh 15` puis `bash resultat.sh 7`.

Plusieurs possibilités : elif

```
note="$1"
if [ "$note" -ge 16 ]; then
echo "Tres bien"
elif [ "$note" -ge 14 ]; then
echo "Bien"
elif [ "$note" -ge 10 ]; then
echo "Admis"
else
echo "Ajourne"
fi
```

Les tests sont évalués dans l'ordre.

Exercice 1 : mentions

Créez `examen.sh` qui reçoit une note et affiche :

- Très bien : note au moins 16 ;
- Bien : note au moins 14 ;
- Assez bien : note au moins 12 ;
- Passable : note au moins 10 ;
- Ajourné : sinon.

Testez les notes 8, 10, 12, 15 et 18.

Tester un fichier

```
fichier="$1"
if [ -f "$fichier" ]; then
echo "Fichier trouve"
else
echo "Fichier introuvable"
fi
```

`-f` teste si le chemin désigne un fichier ordinaire.

Tests courants

Test	Signification
`-e`	chemin existant
`-f`	fichier ordinaire
`-d`	répertoire
`-r`	lecture autorisée
`-w`	écriture autorisée
`-x`	exécution autorisée

Vérifier les arguments

```
if [ "$#" -ne 1 ]; then
echo "Usage : $0 FICHIER"
exit 1
fi
echo "Fichier : $1"
```

- $0 : nom du script ;
- $# : nombre d'arguments ;
- `exit 1` : quitter avec un code d'erreur.

Exercice 2 : vérifier avant de lancer

Écrivez `verifier.sh` qui :

1. exige exactement un argument ;
2. vérifie que le fichier indiqué existe ;
3. affiche une erreur et s'arrête sinon ;
4. lance le fichier avec `python3`.

Exemple : `./verifier.sh programme.py`

Pourquoi une boucle ?

On souhaite lancer un programme plusieurs fois ou traiter de nombreux fichiers.

Principe

Une boucle permet de répéter un bloc d'instructions sans le recopier.

Deux formes seront étudiées : `for` et `while`.

Première boucle for

```
for nombre in 1 2 3 4 5
do
echo "Nombre : $nombre"
done
```

À chaque passage, la variable reçoit la valeur suivante.

Générer une séquence

```
seq 1 5

for i in $(seq 1 10)
do
echo "Iteration $i"
done
```

`$(...)` remplace une commande par sa sortie.

Parcourir les fichiers Python

```
for fichier in *.py
do
echo "Fichier : $fichier"
done
```

Le motif `*.py` désigne les noms terminés par `.py`.

Lancer plusieurs programmes

```
for fichier in *.py
do
if [ -f "$fichier" ]; then
echo "Execution de $fichier"
python3 "$fichier"
fi
done
```

Prudence

Ne lancez que des programmes dont vous connaissez le contenu.

Exercice 3 : répétition

Créez `repetition.sh`.

1. Premier argument : fichier Python.
2. Deuxième argument : nombre d'exécutions.
3. Vérifiez l'existence du fichier.
4. Exécutez le programme le nombre de fois demandé.
5. Affichez le numéro de chaque exécution.

Exemple : `./repetition.sh programme.py 5`

La boucle while

```
i=1
while [ "$i" -le 5 ]
do
echo "Iteration $i"
i=$((i + 1))
done
```

La condition est testée avant chaque itération.

Arithmétique entière

```
a=10
b=3
echo "$((a + b))"
echo "$((a - b))"
echo "$((a * b))"
echo "$((a / b))"
```

Attention

La division entière ne conserve pas les décimales.

Éviter les boucles infinies

```
i=1
while [ "$i" -le 5 ]
do
echo "$i"
# Oubli de la mise a jour de i
done
```

La condition ne devient jamais fausse. Utilisez **Ctrl+C** pour interrompre le programme.

Exercice 4 : compte à rebours

Écrivez `compte_rebours.sh` qui reçoit un entier positif et affiche un compte à rebours jusqu'à zéro.

Exemple

`./compte_rebours.sh 5` affiche 5, 4, 3, 2, 1, 0.

Contrainte : utilisez une boucle `while`.

Projet : automatiser des analyses

Le projet contient :

```
projet/  analyse.py  run.sh
donnees/  janvier.txt  fevrier.txt  mars.txt
```

Chaque fichier texte contient un nombre par ligne. Le programme Python est fourni.

Programme Python fourni

```
import sys
fichier = sys.argv[1]
with open(fichier, encoding="utf-8") as f:
valeurs = [
float(ligne.strip())
for ligne in f
if ligne.strip()
]
print(f"Fichier : {fichier}")
print(f"Nombre : {len(valeurs)}")
print(f"Somme : {sum(valeurs)}")
```

Projet : partie 1

Écrivez `run.sh` pour :

1. vérifier qu'un répertoire est fourni ;
2. vérifier que ce répertoire existe ;
3. parcourir ses fichiers `.txt` ;
4. afficher le nom de chaque fichier ;
5. lancer `analyse.py` avec son chemin ;
6. afficher un message final.

Commande attendue : `./run.sh donnees`

Projet : partie 2

Ajoutez un compteur de fichiers traités.

Sortie attendue

```
Début des analyses
Analyse de janvier.txt
Analyse de fevrier.txt
Analyse de mars.txt
3 fichiers traités
Fin des analyses
```

Le compteur doit évoluer à chaque fichier traité.

Challenge : gestion des erreurs

Améliorez le script pour gérer :

- aucun argument ;
- un répertoire inexistant ;
- un répertoire sans fichier `.txt` ;
- l'échec d'un programme Python.

Objectif

Un script robuste doit réagir correctement aux erreurs.

Synthèse

`if / elif / else`	choisir une action
`-f / -d`	tester un chemin
`$#`	compter les arguments
`for`	parcourir des valeurs
`while`	répéter sous condition
`$((...))`	calculer sur des entiers
`exit`	terminer un script

Suite

Redirections, entrées/sorties et pipelines.